University of Rajshahi
Department of Computer Science and Engineering
CSE3241: Operating Systems
Final Assignment, Submission Deadline: **17-06-2025**

Submission Type:

- Individual Submission
- Hand Written
- Source need to be disclosed

Warning!!!!

- Two submissions cannot have more than 10% similarity

1.	Say there is an exe file named “Addition”. What happens when: A. A single user does double click on its file icon for a single time? B. A single user does double click on its file icon for ‘n’ times? C. Multiple users do separate double clicks on its file icon almost the same time? D. A single user presses ENTER after typing ./Addition in a terminal. Consider program-vs-process concept, memory layout, role of OS loader while answering these questions.	04
2.	Explain when executing the following two instructions belonging to two processes running in parallel is problematic? i++; and i--; when i is in shared memory.	10
3.	Illustrate three cases of race conditions.	15
4.	Show that, if the wait() and signal() semaphore operations are not executed atomically, then mutual exclusion may be violated.	07
	Assume that a system has multiple processing cores. For each of the following scenarios, describe which is a better locking mechanism—a spinlock or a mutex lock where waiting processes sleep while waiting for the lock to become available: • The lock is to be held for a short duration. • The lock is to be held for a long duration. • A thread may be put to sleep while holding the lock	08
5.	Explain why implementing synchronization primitives by disabling interrupts is not appropriate in a single-processor system if the synchronization primitives are to be used in user-level programs.	07
6.	Explain why interrupts are not appropriate for implementing synchronization primitives in multiprocessor systems.	08

7.	What happens in the scenarios shown in Fig. A, Fig. B, Fig. C and Fig.D?	05

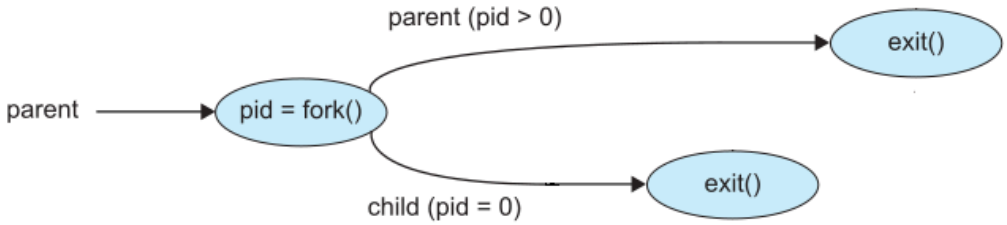


Fig. A

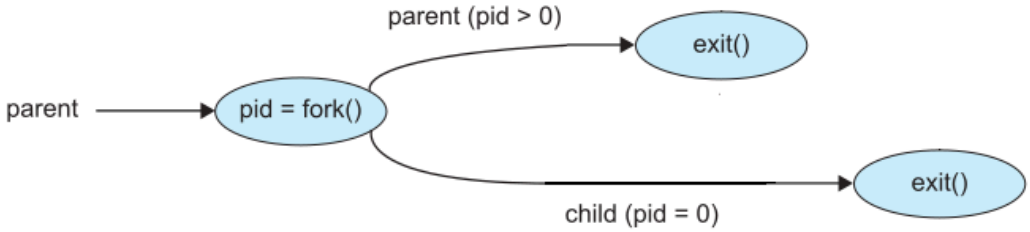


Fig. B

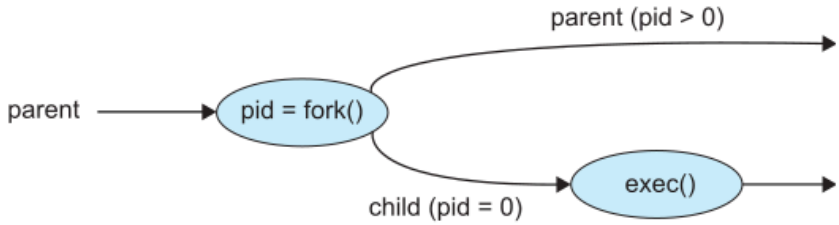


Fig. C

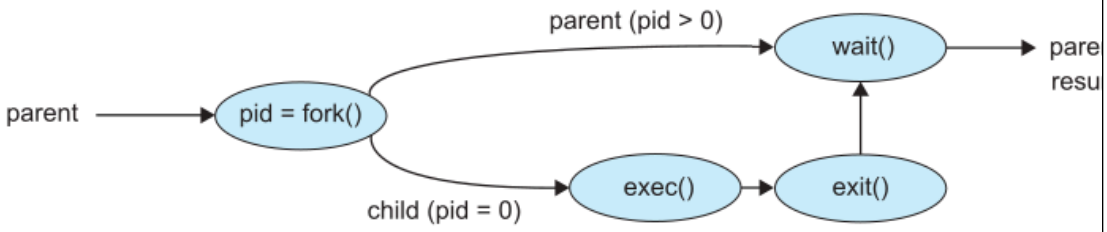


Fig. D

8.	Explain why Windows and Linux implement multiple locking mechanisms. Describe the circumstances under which they use spinlocks, mutex locks, semaphores, and condition variables. In each case, explain why the mechanism is needed.	03

9.	Can a system detect that some of its threads are starving? If you answer “yes,” explain how it can. If you answer “no,” explain how the system can deal with the starvation problem.	03
10.	Can you kill processes having PID 0 and PID 1? If yes, write down the steps of killing these processes.	03

11.	Consider the following code segment:
	`pid_t pid;`
	`pid = fork();`
	`if (pid == 0) {`
	`  fork();`
	`  pthread_create(&tid, NULL,`
	`(void)thread_handler, NULL);`
	`}`
	`fork();`
	
	A. How many unique processes will be created?
	B. How many unique threads will be created?
		02

12.	What will be the output for the following code segment?
	`fork();`
	`printf('Bangladesh');`
	`fork();`
	`printf('Bangladesh');`
	`fork();`
		02

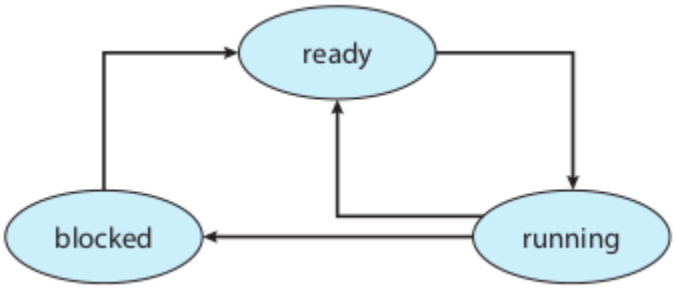
13.	● How many child processes will be created if the following loop is in a program code? Give a proper explanation for your answer.
	`for (i = 0; i < n; i++)`
	`  fork();`
		02

	● Draw the process tree when n = 4.	
14.	Consider a computer system that runs 5,000 jobs per month and has no deadlock-prevention or deadlock-avoidance scheme. Deadlocks occur about twice per month, and the operator must terminate and rerun about ten jobs per deadlock. Each job is worth about two dollars (in CPU time), and the jobs terminated tend to be about half done when they are aborted. A systems programmer has estimated that a deadlock-avoidance algorithm (like the Banker's algorithm) could be installed in the system with an increase of about 10 percent in the average execution time per job. Since the machine currently has 30 percent idle time, all 5,000 jobs per month could still be run, although turnaround time would increase by about 20 percent on average. ● What are the arguments for installing the deadlock-avoidance Algorithms? ● What are the arguments against installing the deadlock-avoidance algorithm?	10
15.	Write down the difference between: i. unnamed pipe and named pipe ii. named pipe and regular file iii. program and process iv. user mode and kernel mode	04
16.	● How many processes can be in a running state, waiting state, ready state at a time? ● Who can have access to the Process Control Block of a process? Justify your answer. ● Where are PCBs stored? Justify your answer. ● Without PCBs, is it possible to handle multiple processes? Justify your answer.	04
17.	Do we need an operating system for all the cases? Justify your answer. A. A single user wants to do a single task using a computer system having a finite set of computer hardware. B. A single user wants to do multiple tasks simultaneously using a computer system having a finite set of computer hardware. C. A single user wants to do multiple tasks consecutively using a computer system having a finite set of computer hardware. D. A single user wants to do a single task using a single hardware. E. Multiple users want to do multiple tasks simultaneously using a computer system having a finite set of computer hardware.	05

18.	Draw the memory layout for the following C program.
	
	``
	#include <stdio.h>
	#include <stdlib.h>
	
	int x, y = 10;
	
	void f2(){
	  int x, y = 10;
	  int *ptr;
	  ptr = (int) malloc(sizeof(int) *5);
	}
	
	void f1(){
	  int x, y = 10;
	  int *ptr;
	  ptr = (int) malloc(sizeof(int) *5);
	  f2();
	}
	
	int main(int argc, char *argv[]){
	  int x, y = 10;
	  int *ptr;
	  ptr = (int) malloc(sizeof(int) *5);
	  return 0;
	}
	``
		10

19.	What can be done to increase the degree of multiprogramming?	01
20.	Explain the journey of a source code (say Addition.c) to turn into a process by the help of a figure.	05

21.	I. What do you know about A. Orphan process B. Zombie process II. When can you call a process an orphan process or a zombie process? III. Can you create them? If yes, describe your steps with code. IV. How can we trace an orphan and zombie process in a Linux based OS? V. Explain how a Linux based OS treats them. VI. What are the advantages and disadvantages of having orphan and zombie processes?	06
22.	How can a process send a signal to A. another process? B. a specific thread of the same process? C. a specific thread of a different process?	03
23.	What happens if a parent process and its child process try to kill each other by sending a SIGKILL signal? Consider as many scenarios as possible.	04
24.	In Linux, how many signals can be caught and cannot be caught by a user-defined signal handler? Make lists of these two categories signals.	03
25.	Write down the differences between: A. user thread and kernel thread. B. child process and thread	04
	Consider a multicore system and a multithreaded program written using the many-to-many threading model. Let the number of user-level threads in the program be greater than the number of processing cores in the system. Discuss the performance implications of the following scenarios. A. The number of kernel threads allocated to the program is less than the number of processing cores. B. The number of kernel threads allocated to the program is equal to the number of processing cores. C. The number of kernel threads allocated to the program is greater than the number of processing cores but less than the number of user-level threads.	06
26.	In Linux, if one thread in a program calls fork() , does the new process duplicate all threads, or is the new process single-threaded?	03
27.	Is the Linux kernel multithreaded? Justify your answer.	04

28.	Imagine that a server needs to serve 1000 clients. Which way would be beneficial for the server to serve clients: A. put service instructions in the main thread of the server process. B. create a separate child process to serve each client. C. create a separate thread to serve each client. D. follow a hybrid structure: at first create a few child processes where each child process will have multiple threads and multiple child processes so that each leaf child process and thread is dedicated to serve a single client. Justify your answer by comparing four cases.	10
29.	What are the reasons behind using separate registers, stack and program counter for a multi-threaded process?	04
30.	Do multiple child processes face any problems if they try to update: ● Local variables ● Global variables Explain your answer as much as possible.	05
31.	Do multiple threads face any problems if they try to update: ● Local variables ● Global variables Explain your answer as much as possible.	05
32.	What happens when main thread terminates before termination of: ● child processes ● (subordinate) threads	04
33.	Give five examples when multithreading is beneficial.	05
34.	A simplified view of thread states is ready, running, and blocked, where a thread is either ready and waiting to be scheduled, is running on the processor, or is blocked (for example, waiting for I/O).  	02
35.	What do you know about: I. Process II. Process Control Block (PCB)	07

	III. PCB Table IV. Process ID (PID) V. Process Tree VI. Processes having PID 0 and PID 1.	
36.	Write down the differences between the following terms: I. Preemptive and Non-Preemptive Scheduling Algorithms II. User mode and kernel mode III. Named Pipe and Unnamed Pipe IV. Conventional File and Named Pipe V. Logical Core and Physical Core VI. Default Signal Handler and User-Defined Signal Handler	06
37.	Write down the positive and negative sides of the following scheduling algorithms I. Shortest-Job-First Scheduling II. First-Come, First-Served Scheduling III. Round-Robin Scheduling IV. Priority Scheduling V. Preemptive Scheduling VI. Non-Preemptive Scheduling	03
38.	Which scheduling algorithms could result in starvation? How can the OS solve it?	02
39.	When instructions after execlp() are executed? Why?	02
40.	Can an operating system detect that some of its processes are starving? Justify your answer.	1.5
41.	A. How many ways can you establish communication between a parent process and its child process? B. Which way do you prefer? Justify your answer by pointing at least one positive point for your preferred way and at least one negative point for your non-preferred way.	
42.	Assume that two processes (P0 and P1) are currently at running state at a multi-core machine. Figure out pid's values in question mark (?) position. Which scenario is problematic? Why?	3.0

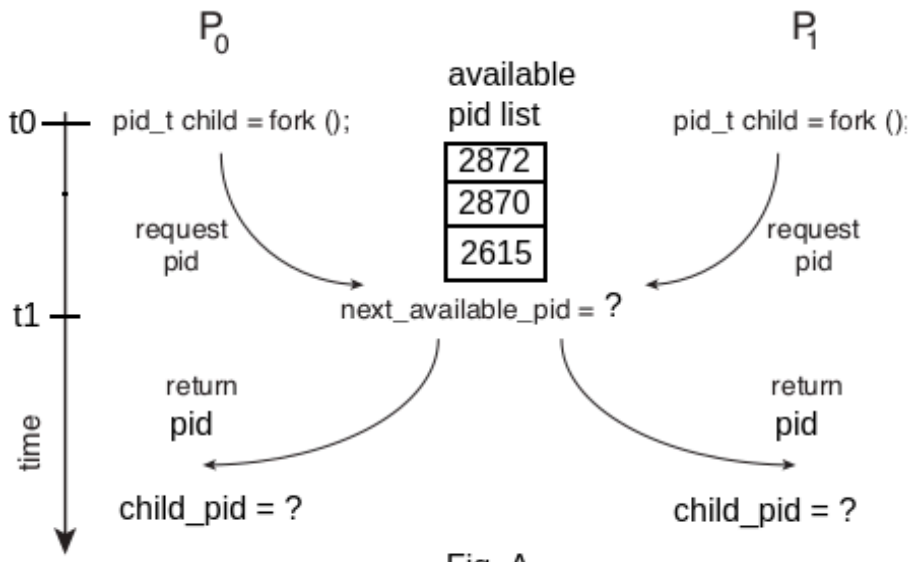


Fig. A

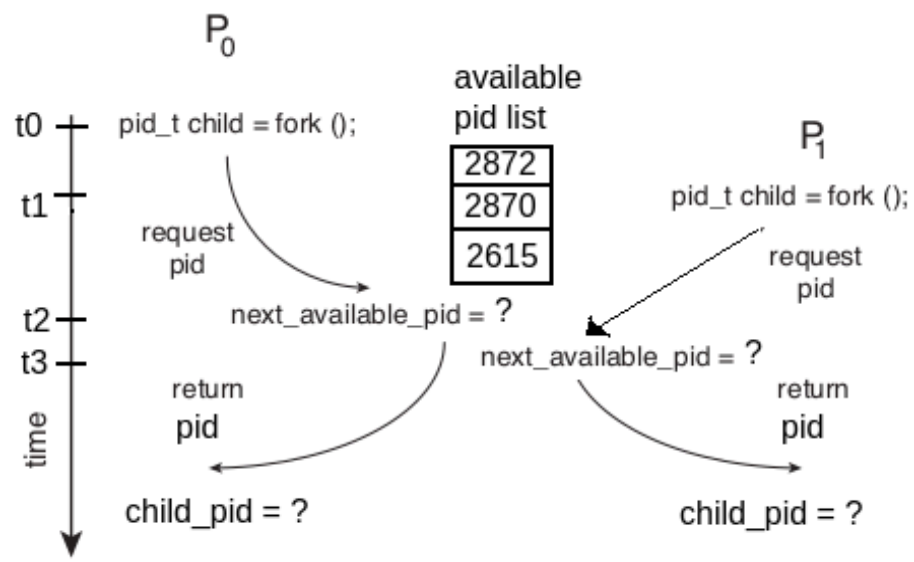
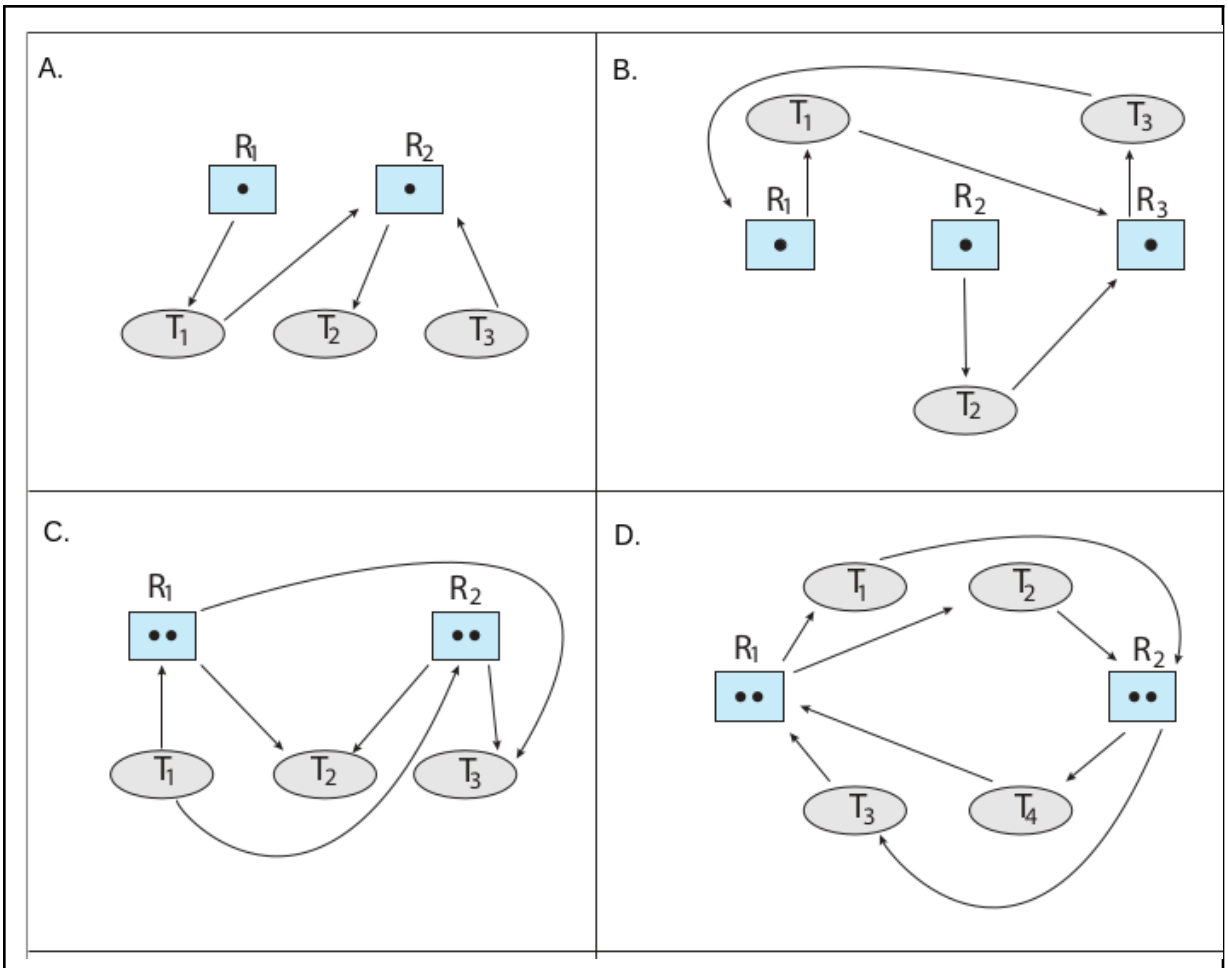


Fig. B

43.	Can two non-related processes communicate with each other via an unnamed pipe? Justify your answer.	02
44.	Which of the four resource-allocation graphs shown in the following illustrate deadlock? For those situations that are deadlocked, provide the cycle of threads and resources. Where there is not a deadlock situation, illustrate the order in which the threads may complete execution.	04



45.	What happens when local and global variables are updated by: A. multiple child processes belonged to a single parent process, B. multiple threads belonged to a single process? Justify your answer.	03
46.	Consider the following tasks of an operating system. Rank these tasks on a seven-point scale from “the most important” to “the least important” based on your understanding. i. Process Scheduling ii. Process Synchronization iii. Multi-Thread Handling iv. Deadlock handling v. Main Memory Management vi. Virtual Memory Management vii Mass Storage Management Justify your answer.	06
47.	Which illustration(s) is/are problematic? Justify your answer.	02

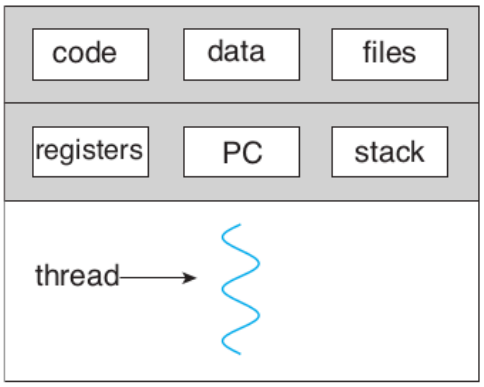


Fig. I single-threaded process

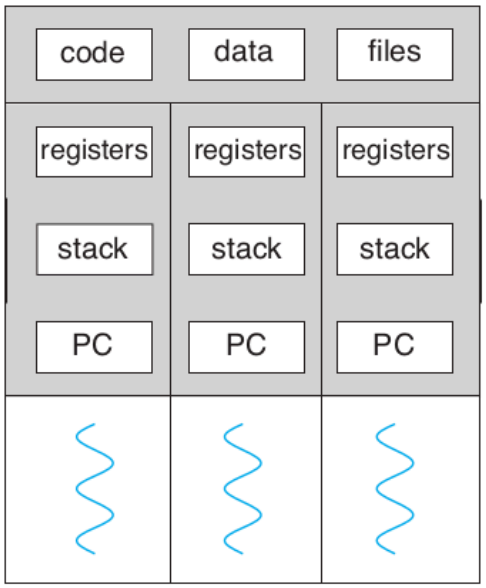


Fig. II multithreaded process

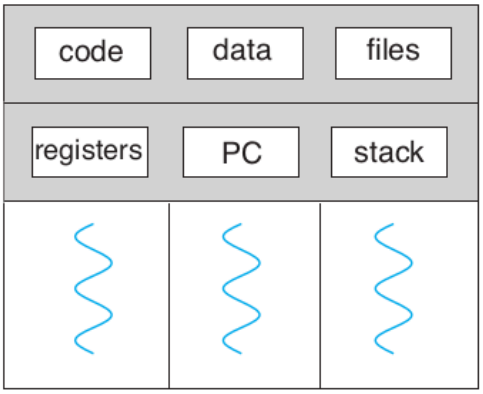


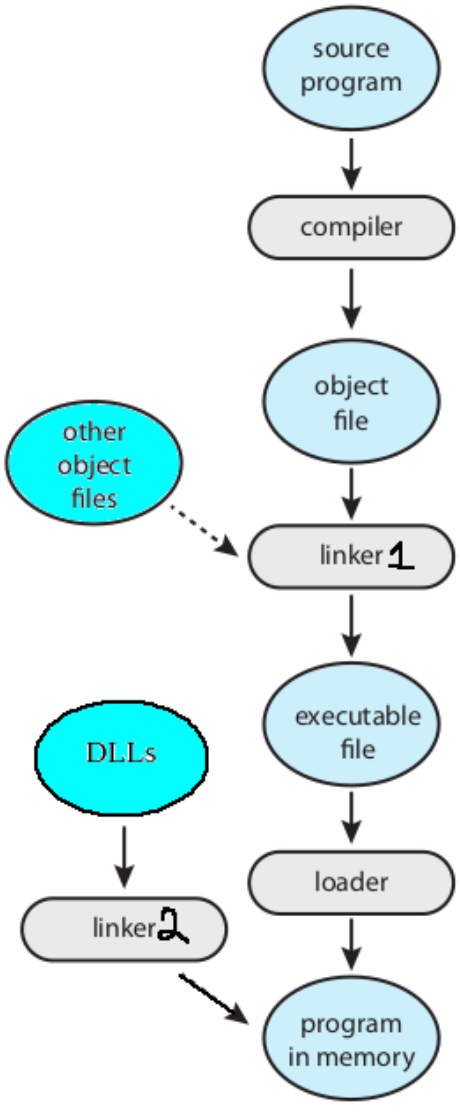
Fig. III multithreaded process

PC: Program Counter

48.	A multithreaded web server wishes to keep track of the number of requests it services (known as hits). Consider the two following strategies to prevent a race condition on the variable hits . The first strategy is to use a basic mutex lock when updating hits :
	`int hits;`
	`mutex_lock hit_lock;`
	`hit_lock.acquire();`
	`hits++;`
	`hit_lock.release();`
	A second strategy is to use an atomic integer:
	`atomic_t hits;`
	`atomic_inc(&hits);`
	Explain which of these two strategies is more efficient.
		15

49.	What do you understand by ‘’?	

50.	I. What do you understand by ‘semaphore’? II. Write down one use case of a binary semaphore and a counter semaphore.	02
51.	Write down the works done by: ● Compiler ● Linker-1 ● Loader ● Linker-2 Among them which needs to be part of the operating system? Why?	



52.	Imagine that a server needs to serve 1000 clients. Which way would be beneficial for the server to serve clients: A. put service instructions in the main thread of the server process. B. create a separate child process to serve each client.

	C. create a separate thread to serve each client. D. follow a hybrid structure: at first create a few child processes where each child process will have multiple threads and multiple child processes so that each leaf child process and thread is dedicated to serve a single client. Justify your answer by mentioning at least one negative point for non-beneficial ways.